

JAVA PROGRAMMING MASTERCLASS

包裝類別

「讓基本型態穿上物件的外衣，解鎖完整功能」

[← 返回目錄](#)

Outline

- 包裝類別概念與對應關係
- 自動裝箱與自動拆箱 (**Autoboxing / Unboxing**)
- **Integer** 類別的常用方法
- 各類型的包裝類別方法
- 實作練習

第一部分

包裝類別概念

什麼是包裝類別？

Java 的資料型態分為兩種：

- 基本型態 (Primitive type) : `int` , `double` , `boolean` ... 儲存純值，放在 Stack
- 參考型態 (Reference type) : 物件，放在 Heap，可以呼叫方法

```
1  int score = 95;           // 基本型態，無法呼叫方法
2  Integer scoreObj = 95;    // 包裝類別，可以呼叫方法
```

💡 包裝類別讓每種基本型態都有對應的「物件版本」，擁有各種工具方法可供使用。

基本型態與包裝類別對照表

基本型態

包裝類別

byte

Byte

short

Short

int

Integer

long

Long

float

Float

double

Double

char

Character

boolean

Boolean

第二部分

自動裝箱與自動拆箱

手動裝箱 (Boxing)

將基本型態轉換成包裝類別物件，稱為「裝箱」：

```
1 // 手動裝箱 (JDK 5 以前的舊寫法)
2 int hp = 100;
3 Integer hpObj = Integer.valueOf(hp);
4
5 System.out.println(hpObj); // 100
```

自動裝箱 (Autoboxing)

Java 5 之後，編譯器自動完成裝箱：

```
1  int hp = 100;  
2  Integer hpObj = hp; // 編譯器自動呼叫 Integer.valueOf(hp)
```

💡 自動裝箱讓基本型態可以直接指派給包裝類別變數，背後由編譯器自動呼叫 `valueOf()` 。

自動拆箱 (Unboxing)

從包裝類別物件取出基本型態，稱為「拆箱」：

```
1  Integer hpObj = 100;
2
3  // 自動拆箱：編譯器呼叫 hpObj.intValue()
4  int hp = hpObj;
5
6  // 也可直接參與數學運算
7  int newHp = hpObj + 50; // 自動拆箱後計算
8  System.out.println(newHp); // 150
```

自動裝拆箱與集合框架

包裝類別最常用的場景：放入只接受物件的集合框架

```
1  import java.util.ArrayList;
2
3  ArrayList<Integer> ranks = new ArrayList<>();
4
5  // 自動裝箱：int 放入 ArrayList
6  ranks.add(1);    // 等同 ranks.add(Integer.valueOf(1))
7  ranks.add(3);
8
9  // 自動拆箱：Integer 取出後可直接計算
10 int total = ranks.get(0) + ranks.get(1); // 4
```


裝箱陷阱：== 比較的問題

包裝類別是物件，`==` 比較的是位址，而非數值：

```
1 Integer a = 200;
2 Integer b = 200;
3 System.out.println(a == b);      // false (不同物件)
4 System.out.println(a.equals(b)); // true  (數值相同)
```

⚠ 快取範圍：Java 快取 -128 到 127 的 Integer 物件，在此範圍內 `==` 可能為 true，超出則為 false。建議永遠用 `equals()`。

null 陷阱：自動拆箱的危險

當包裝類別為 `null` 時，自動拆箱會觸發 `NullPointerException`：

```
1 Integer hp = null;
2
3 // 以下會拋出 NullPointerException!
4 int damage = hp - 10;
```

```
1 // 安全寫法：先判斷 null
2 int damage = (hp != null) ? hp - 10 : 0;
```


第三部分

Integer 類別的常用方法

Integer 常數

Integer 類別提供了重要的邊界常數：

```
1 System.out.println(Integer.MAX_VALUE); // 2147483647
2 System.out.println(Integer.MIN_VALUE); // -2147483648
3 System.out.println(Integer.SIZE);      // 32 (位元數)
4 System.out.println(Integer.BYTES);     // 4 (位元組數)
```

💡 int 的範圍是 -2^{31} 到 $2^{31}-1$ ，超出此範圍會發生「整數溢位 (overflow)」，需改用 **long** 或 **Long**。

Integer 型態轉換方法

方法名稱

說明

`Integer.parseInt(String s)`

字串轉 `int`

`Integer.valueOf(int i)`

`int` 轉 `Integer`

`Integer.valueOf(String s)`

字串轉 `Integer`

`intValue()`

`Integer` 轉 `int` (拆箱)

`toString()`

`Integer` 轉字串

字串轉整數 — 範例

```
1  String inputHp = "150";
2
3  // 字串 → int (最常用)
4  int hp = Integer.parseInt(inputHp);
5
6  // 字串 → Integer
7  Integer hpObj = Integer.valueOf(inputHp);
8
9  System.out.println(hp + 50);    // 200
10 System.out.println(hpObj + 50); // 200
```


進位轉換方法

方法名稱	說明
<code>Integer.toString(int i)</code>	轉換成二進位字串
<code>Integer.toOctalString(int i)</code>	轉換成八進位字串
<code>Integer.toHexString(int i)</code>	轉換成十六進位字串
<code>Integer.parseInt(String s, int radix)</code>	將指定進位的字串轉為十進位

進位轉換 – 範例

```
1  int power = 255;
2
3  System.out.println(Integer.toBinaryString(power)); // 11111111
4  System.out.println(Integer.toOctalString(power));  // 377
5  System.out.println(Integer.toHexString(power));    // ff
6
7  // 二進位字串 "11111111" 轉回十進位
8  System.out.println(Integer.parseInt("11111111", 2)); // 255
```


數值比較與工具方法

方法名稱	說明
<code>Integer.max(int a, int b)</code>	回傳較大值
<code>Integer.min(int a, int b)</code>	回傳較小值
<code>Integer.sum(int a, int b)</code>	回傳總和
<code>Integer.compare(int x, int y)</code>	比較大小，回傳負/零/正

數值比較 – 範例

```
1  int atkA = 180; // 炭治郎攻擊力
2  int atkB = 160; // 禰豆子攻擊力
3
4  System.out.println(Integer.max(atkA, atkB)); // 180
5  System.out.println(Integer.min(atkA, atkB)); // 160
6  System.out.println(Integer.sum(atkA, atkB)); // 340
7  System.out.println(Integer.compare(atkA, atkB)); // 正值 (atkA > atkB)
```


第四部分

其他包裝類別方法

Double 類別

方法名稱	說明
<code>Double.parseDouble(String s)</code>	字串轉 <code>double</code>
<code>Double.isNaN(double v)</code>	是否為 NaN (非數值)
<code>Double.isInfinite(double v)</code>	是否為無限大
<code>Double.max(double a, double b)</code>	回傳較大值

```
1 System.out.println(Double.parseDouble("3.14")); // 3.14
2 System.out.println(Double.isNaN(0.0 / 0.0));    // true
3 System.out.println(Double.isInfinite(1.0 / 0.0)); // true
```


Boolean 類別

方法名稱	說明
<code>Boolean.parseBoolean(String s)</code>	字串轉 <code>boolean</code>
<code>Boolean.valueOf(boolean b)</code>	<code>boolean</code> 轉 <code>Boolean</code>
<code>Boolean.toString(boolean b)</code>	<code>boolean</code> 轉字串
<code>Boolean.compare(boolean x, boolean y)</code>	比較大小 (true > false)

```
1 System.out.println(Boolean.parseBoolean("true")); // true
2 System.out.println(Boolean.parseBoolean("True")); // true
3 System.out.println(Boolean.parseBoolean("yes"));  // false
```


Character 類別（複習整合）

Character 是字元型態 `char` 的包裝類別，常用方法複習：

方法名稱	說明
<code>Character.isDigit(char ch)</code>	是否為數字字元
<code>Character.isLetter(char ch)</code>	是否為字母字元
<code>Character.toUpperCase(char ch)</code>	轉大寫
<code>Character.toLowerCase(char ch)</code>	轉小寫

💡 Character 類別在 Ch12 已詳細介紹，這裡複習其作為包裝類別的定位。

Number 抽象父類別

Integer 、 **Double** 、 **Float** 、 **Long** 等數值型別的共同父類別：

```
1  Number n1 = Integer.valueOf(42);
2  Number n2 = Double.valueOf(3.14);
3
4  System.out.println(n1.intValue());    // 42
5  System.out.println(n2.doubleValue()); // 3.14
6  System.out.println(n2.intValue());    // 3 (截斷小數)
```

💡 **Number** 類別定義了 **intValue()** 、 **doubleValue()** 、 **longValue()** 等方法，所有數值型包裝類別都繼承自它。

練習一：讀取使用者輸入並計算

任務說明

請利用 **Scanner** 讀取兩個整數字串，轉換後計算並印出：

1. 兩數的和、差、積
2. 哪個數比較大

```
1 Scanner sc = new Scanner(System.in);  
2 String s1 = sc.nextLine();  
3 String s2 = sc.nextLine();  
4 // 請利用 Integer 的方法完成轉換與計算
```

練習一：解題提示

1. 使用 `Integer.parseInt()` 將字串轉為 `int`。
2. 用算術運算子計算和、差、積。
3. 使用 `Integer.max()` 找出較大值。

練習二：判斷字串是否為合法整數

任務說明

撰寫一個方法 `isInteger(String s)`，判斷輸入的字串是否可以成功轉為整數。

- `"123"` → true
- `"-45"` → true
- `"12.3"` → false
- `"abc"` → false

練習二：解題提示

```
1  static boolean isInteger(String s) {  
2      try {  
3          Integer.parseInt(s);  
4          return true;  
5      } catch (NumberFormatException e) {  
6          return false;  
7      }  
8  }
```


練習三：進位制轉換器

任務說明

讓使用者輸入一個十進位整數，印出它的二進位、八進位和十六進位表示。

- 輸入：**255**
- 輸出：
 - 二進位：**11111111**
 - 八進位：**377**
 - 十六進位：**ff**

練習三：解題提示

1. 使用 `Integer.parseInt()` 讀取輸入。
2. 呼叫 `Integer.toBinaryString()`。
3. 呼叫 `Integer.toOctalString()`。
4. 呼叫 `Integer.toHexString()`。

Q & A